

Git and GitHub

Dario Pescini

Università degli Studi di Milano-Bicocca

Department of Statistics and Quantitative Methods

dario.pescini@unimib.it

The Problem

Have you ever found yourself doing this?

```
analysis.py
analysis_v2.py
analysis_v2_final.py
analysis_v2_final_FIXED.py
analysis_v2_final_FIXED_actually_final.py
```

The Problem

Have you ever found yourself doing this?

```
analysis.py
analysis_v2.py
analysis_v2_final.py
analysis_v2_final_FIXED.py
analysis_v2_final_FIXED_actually_final.py
```

Questions that become impossible to answer:

- What changed between these versions?
- Which one produced the results in my report?
- Can I recover the version from two weeks ago?

◆ Version Control

A **version control system** records the full history of a project, file by file, change by change.

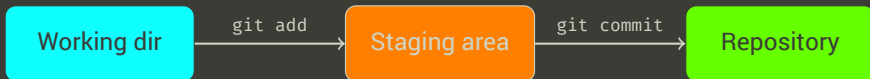
Without VCS

- Multiple copies of files
- No record of what changed or why
- Hard to collaborate
- Recovery depends on luck

◆ With Git

- One copy, full history
- Every change is explained
- Conflicts resolved explicitly
- Any past state is recoverable

◆ Git — Key Concepts



- **Working dir**: where you actually work
- **Staging area**: where you choose what goes into the next commit
- **Commit**: a named snapshot of the staging area
- **Repository (repo)**: a folder whose history Git tracks

◆ First-Time Configuration

Run once on every machine you use:

```
git config --global user.email "you@example.com"  
git config --global core.editor "code --wait"  
git config --global init.defaultBranch main
```

Check your settings at any time:

```
git config --list
```

◆ Starting a Repository

```
cd genome_analysis  
git init
```

Git creates a hidden `.git/` folder — the repository database. Never edit it manually.

```
On branch main  
No commits yet  
nothing to commit
```

◆ Your First Commit

```
git status          # notes.md appears as "untracked"

git add notes.md     # move to staging area
git status           # now "Changes to be committed"

git commit -m "Add initial analysis notes"
```

Write good commit messages

The message is addressed to your future self.

Write **why**, not just **what**.

Good: "Fix off-by-one error in codon frame selection"

Bad: "fix bug"

◆ The Staging Area — Why Does It Exist?

It lets you build a focused, meaningful commit even when several files have changed:

```
# You edited three files:
#  gc_content.py    <- new feature
#  notes.md         <- documentation update
#  scratch.py       <- unfinished experiment
```

```
git add gc_content.py
git add notes.md
git commit -m "Implement GC content function and document it"
```

scratch.py is left out -- it will go in a later commit

Rule of thumb

Each commit should represent one logical change. A reader should understand it from the message alone, without reading the diff.

📖 Viewing History

```
git log          # full log with author, date, message
git log --oneline # compact: one commit per line
git log --oneline -5 # last 5 commits only
```

Example output:

```
a3f2e1c Add GC content function
8b91d04 Add initial analysis notes
```

Each commit has a unique **hash** (the hexadecimal code). This is how you refer to any past state.

◆ Inspecting Changes

```
git diff                # unstaged changes vs last commit
git diff --staged       # staged changes vs last commit
git show a3f2e1c        # what a specific commit changed
```

Output format — lines starting with:

+	added line
-	removed line
(none)	unchanged context

◆ Undoing Changes

```
# Discard unstaged changes in a file
git restore notes.md

# Unstage a file (keep the changes, remove from staging)
git restore --staged notes.md

# Inspect the state at a past commit (read-only)
git checkout a3f2e1c
git checkout main           # return to the present
```

Safe vs caution

```
git restore only touches unstaged or staged files — safe.
git checkout <hash> puts you in “detached HEAD” state — read-only,
return with git checkout main.
```

◆ What to Track — and What Not to

Track

- Source code and scripts
- Documentation and notes
- Configuration files

Do not track

- Large data files
- Generated outputs (PDFs, plots)
- Secrets (passwords, API keys)

Use a `.gitignore` file to exclude patterns:

```
*.pdf  
__pycache__/  
.env  
data/raw/
```

🔄 What Is GitHub?

🔄 GitHub is a **hosting platform for Git repositories**.

What it adds to Git

- Remote backup
- Web interface to browse history
- Pull requests and code review
- Issue tracking
- CI/CD integration

Alternatives

- GitLab (self-hostable)
- Bitbucket
- Codeberg (open source)

Git is the tool. GitHub is one place to store the result.

🔄 Creating a Remote Repository

- 1 Go to github.com → click + → [New repository](#)
- 2 Name it (e.g. `genome_analysis`)
- 3 Choose [Public](#) or [Private](#)
- 4 [Leave all checkboxes empty](#) — you already have a local repo

Public vs Private

- [Public](#) — anyone can read; good for open science and sharing results
- [Private](#) — only you and invited collaborators can see it

🔗 Connecting Local to Remote

Tell your local repo where the remote is:

```
git remote add origin  
git@github.com:youruser/genome_analysis.git  
  
git remote -v          # verify
```

Output:

```
origin  git@github.com:youruser/genome_analysis.git (fetch)  
origin  git@github.com:youruser/genome_analysis.git (push)
```

`origin` is the conventional name for the main remote.

🔄 Pushing and Pulling

First push: `-u` sets `origin/main` as the tracking branch
`git push -u origin main`

All subsequent pushes (after new commits)
`git push`

Fetch remote changes and merge into your local branch
`git pull`

The daily workflow

`git pull` → *edit* → `git add` → `git commit` → `git push`

🔄 Cloning an Existing Repository

To get a copy of any repository — yours or someone else's:

```
git clone git@github.com:youruser/genome_analysis.git
```

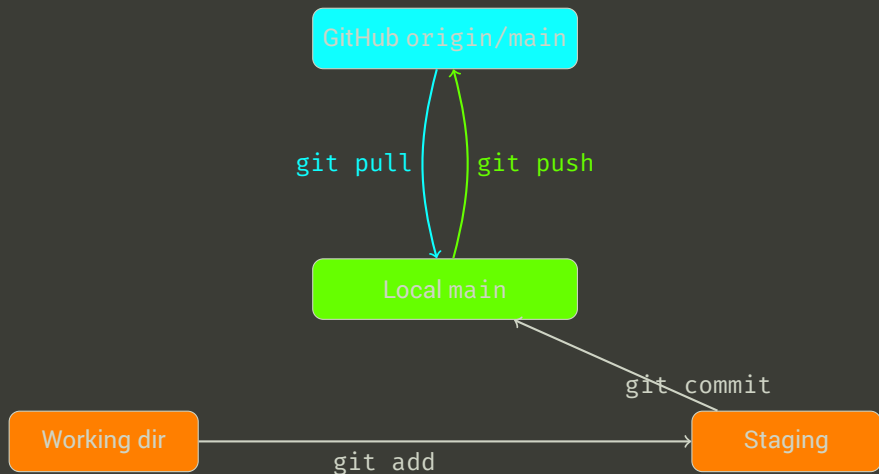
Clone to a custom folder name

```
git clone git@github.com:youruser/genome_analysis.git my_copy
```

`git clone` does everything in one step:

- Creates the folder
- Initialises a Git repo
- Downloads the full history
- Sets up `origin` automatically

🔄 Remote + Local — The Full Picture



Why Branches?

On a shared project, committing directly to `main` causes problems:

- Half-finished work breaks others' code
- Two people editing the same file simultaneously → conflict
- Hard to review changes before they reach the stable version

Branches

Let each person (or each feature) have an **isolated line of development** that only merges into `main` when it is ready.

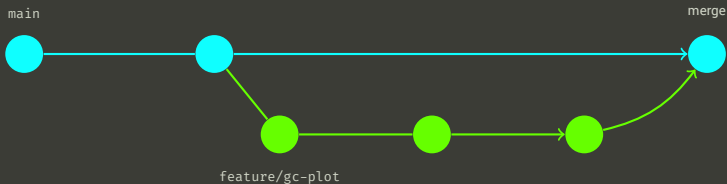
Working with Branches

```
git branch                # list all branches
git checkout -b feature/gc-plot # create and switch
git switch main           # return to main
git branch -d feature/gc-plot # delete a merged branch
```

Naming convention

- main — stable, always works
- feature/gc-plot — new feature in progress
- fix/parser-crash — bug fix
- experiment/new-model — exploratory, may be discarded

◆ The Branch Lifecycle



```
# On feature/gc-plot: develop and commit as usual
git add plot_gc.py
git commit -m "Add GC content plot function"
```

```
# When ready, merge back into main
git checkout main
git merge feature/gc-plot
```

Pull Requests

A **Pull Request (PR)** is a GitHub-level concept: a formal request to merge one branch into another, with a built-in review interface.

Workflow:

- 1 Push your branch: `git push -u origin feature/gc-plot`
- 2 GitHub shows a **Compare & pull request** button — click it
- 3 Write a description of your changes
- 4 Assign reviewers
- 5 Reviewers comment, request changes, or approve
- 6 Merge via GitHub interface

Why PRs Matter for Research

Code review

- Catch bugs before they affect results
- Share knowledge across the team
- Enforce consistent style

Documentation

- Every change has a reason recorded
- Reviewers' comments are preserved
- You know exactly what version produced a figure

Git history is your lab notebook — treat it with the same care.

Merge Conflicts

A conflict happens when two branches edited **the same lines** of the same file.

Git marks the conflict in the file:

```
<<<<<<< HEAD
gc = (seq.count("G") + seq.count("C")) / len(seq)
=====
gc = sum(1 for b in seq if b in "GC") / len(seq)
>>>>>> feature/gc-refactor
```

Resolution steps:

- 1 Open the file, decide which version to keep (or combine them)
- 2 Remove all ««, ==, »» markers
- 3 `git add` the resolved file
- 4 `git commit`

Forking

A **fork** is a personal copy of someone else's repository on GitHub.

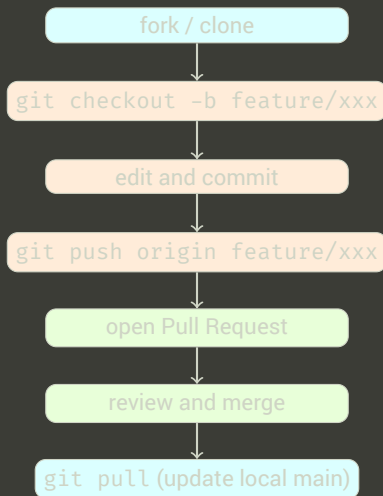
Use it when you want to contribute to a project you do not have write access to:

- 1 Fork the repo on GitHub (click **Fork**)
- 2 Clone **your fork** locally
- 3 Make changes, push to your fork
- 4 Open a PR from your fork to the original repo

In bioinformatics

This is how open-source contributions work — including contributions to Biopython, Snakemake, GATK, and other tools you will use in this course.

The Standard Collaborative Workflow



Commands Reference — Local

<code>git init</code>	Create a new repository
<code>git status</code>	Show current state
<code>git add <file></code>	Stage changes
<code>git commit -m "..."</code>	Save a snapshot
<code>git log --oneline</code>	Show history
<code>git diff</code>	Show unstaged changes
<code>git restore <file></code>	Discard unstaged changes

Commands Reference — Remote and Branches

<code>git remote add origin <url></code>	Connect to a remote
<code>git push</code>	Upload commits to remote
<code>git pull</code>	Download and merge remote commits
<code>git clone <url></code>	Copy a remote repository locally
<code>git checkout -b <name></code>	Create and switch to a branch
<code>git merge <branch></code>	Merge a branch into current

Key Takeaways

- *Commit early and often* — small commits are easier to review and revert
- *Write meaningful messages* — “fix bug” tells you nothing in six months
- *Never commit secrets* — API keys, passwords; use `.gitignore`
- *Pull before you push* — avoid unnecessary conflicts
- *Use branches for anything non-trivial* — keep `main` always working
- *Git history is your lab notebook* — treat it with the same care